

11 장 학습 과제

과제 1. 표준 라이브러리 구성 요소 구분

`std::vector`, `std::sort`, `std::greater`, `std::string`, `std::unique_ptr`, `std::ifstream`, `std::thread`를 컨테이너, 알고리즘, 함수 객체, 문자열, 메모리 관리, 입출력, 동시성으로 분류한다. 각 타입이나 함수가 정의된 헤더와 `std` 네임스페이스 사용 여부를 기록한다. 표준 라이브러리 전체와 STL이라는 표현의 범위 차이도 서술한다.

과제 2. 반복자 범위와 알고리즘

정수 원소 20개를 가진 `std::vector`를 작성하고 표준 알고리즘으로 작업을 수행한다.

- 값 50 검색
- 오름차순 정렬
- 짝수 개수 계산
- 모든 원소의 합 계산
- 30 미만 원소 제거

인덱스 기반 반복문을 사용하지 않고 반복자, 범위 기반 `for`, 표준 알고리즘을 활용한다. `erase()`와 반복자 반환값을 이용한 제거 방식과 C++20 `std::erase_if()` 방식도 비교한다.

과제 3. `std::array`와 C 배열 비교

정수 5개를 저장하는 C 배열과 `std::array<int, 5>`를 작성한다. 두 배열에 대해 크기 확인, 함수 인수 전달, 전체 복사, 비교, 정렬 코드를 작성한다.

C 배열이 함수 인수에서 포인터로 변환될 때 크기 정보가 어떻게 처리되는지 설명한다.

여러 크기의 `std::array`를 받을 수 있는 함수 템플릿도 구현한다.

과제 4. `vector`의 크기와 용량 관찰

빈 `std::vector<int>`에 원소 100개를 하나씩 추가하면서 `size()`, `capacity()`, `data()` 주소가 변한 시점을 출력한다.

`reserve(100)`을 호출한 경우와 호출하지 않은 경우를 비교한다. 용량 증가 배율이 표준으로 고정되어 있지 않다는 점을 기록하고, 재할당 횟수와 주소 변경 횟수를 측정한다.

과제 5. `reserve`와 `resize` 구분

`std::vector<int>`에 `reserve(10)`과 `resize(10)`을 각각 적용하고 `size()`, `capacity()`, 반복 가능한 원소 수를 비교한다.

생성자와 소멸자 호출 횟수를 출력하는 `Trace` 클래스를 원소 타입으로 사용하여 두 함수가 객체 생성에 미치는 영향을 확인한다.

과제 6. 반복자 무효화 확인

`std::vector<int>`의 첫 원소를 가리키는 반복자, 포인터, 참조를 저장한다. 원소 추가 전후의 `capacity()`와 `data()`를 출력하여 재할당 여부를 확인한다.

무효화된 반복자를 실제로 역참조하지 않고 주소와 용량 변화만으로 사용 가능 여부를 판단한다. `reserve()`를 적용한 경우와 적용하지 않은 경우를 비교한다.

같은 실험을 `std::list<int>`에서도 수행하여 삽입 후 기존 반복자가 유지되는지 확인한다.

과제 7. `vector`와 `deque` 비교

앞과 뒤에 정수를 추가하고 제거하는 작업을 `std::vector`와 `std::deque`로 각각 구현한다.

벡터에서 앞쪽 삽입이 원소 이동을 발생시키는 이유와 덱이 양쪽 끝 작업을 효율적으로 처리하는 이유를 저장 구조와 함께 설명한다. 두 컨테이너가 임의 접근을 지원하지만 `data()` 제공 여부가 다른 이유도 기술한다.

과제 8. `list`와 `forward_list` 조작

`std::list<std::string>` 두 개를 만들고 `splice()`로 한 리스트의 일부 원소를 다른 리스트로 옮긴다. 원소 복사 없이 노드 연결이 변경되는 것을 반복자와 주소 출력으로 확인한다.

`std::forward_list<int>`에서는 `before_begin()`, `insert_after()`, `erase_after()`를 사용하여 첫 위치 삽입과 제거를 구현한다. `size()`와 `push_back()`이 없는 구조적 이유를 설명한다.

과제 9. 문자열 파싱

쉼표로 구분된 문자열을 입력받아 이름, 레벨, 체력으로 분리한다.

`Knight,12,150`

`find()`와 `substr()`로 각 필드를 추출하고 숫자 필드는 `std::stoi()`로 변환한다. 구분자 누락, 숫자 변환 실패, 남은 문자가 있는 숫자 필드를 검사한다.

같은 숫자 변환을 `std::from_chars()`로 구현하고 예외 처리 방식과 오류 코드 검사 방식을 비교한다.

과제 10. 순차 컨테이너 선택 보고서

제시된 기능마다 적합한 순차 컨테이너를 선택하고 근거를 작성한다.

- 항상 세 좌표를 저장하는 위치 데이터
- 실행 중 계속 추가되는 로그 목록
- 앞과 뒤에서 작업을 넣고 빼는 작업 대기열
- 원소 주소가 유지되어야 하고 리스트 사이 이동이 잦은 객체 목록
- 고정 길이가 아닌 사용자 입력 문자열
- 대량 숫자를 외부 C 함수에 전달하는 버퍼

각 선택에서 크기 변경, 연속 메모리, 임의 접근, 삽입과 제거 위치, 반복자 안정성, 메모리 부가 비용을 평가한다. `std::vector`가 부적합하다고 판단한 항목은 벡터가 충족하지 못하는 요구를 구체적으로 밝힌다.